CONTRACT WOLF

*Security Assessment*

# Veil

Verified on October 17, 2026

## SUMMARY

| Project | CHAIN | METHODOLOGY |
|---|---|---|
| Veil | Solana | Manual & Automatic Analysis |

| FILES | DELIVERY | TYPE |
|---|---|---|
| Browser Extension Source | October 17, 2026 | Standard Audit |



| 10 | 0 | 1 | 1 | 5 | 3 | 10 |
|---|---|---|---|---|---|---|
| Total Findings | Critical | Major | Medium | Minor | Informational | Resolved |

| | |
|---|---|
| 0 Critical | An exposure that can affect the contract functions in several events that can risk and disrupt the contract |
| 1 Major | An opening & exposure to manipulate the contract in an unwanted manner |
| 1 Medium | An opening that could affect the outcome in executing the contract in a specific situation |
| 5 Minor | An opening but doesn't have an impact on the functionality of the contract |
| 3 Informational | An opening that consists information but will not risk or affect the contract |
| 10 Resolved | ContractWolf's findings has been acknowledged & resolved by the project |

**STATUS** ✔ AUDIT PASSED

# TABLE OF CONTENTS | Veil

## Summary

Project Summary
Findings Summary
Disclaimer
Scope of Work
Auditing Approach

## Project Information

Token/Project Details

## Findings

Issues
Rust Attacks
Fixes & Recommendation
Audit Comments

# DISCLAIMER | Veil

**ContractWolf** audits and reports should not be considered as a form of project's "Advertisement" and does not cover any interaction and assessment from "Project Contract" to "External Contracts" such as PancakeSwap, UniSwap, SushiSwap or similar.

**ContractWolf** does not provide any warranty on its released report and should not be used as a decision to invest into audited projects.

**ContractWolf** provides a transparent report to all its "Clients" and to its "Clients Participants" and will not claim any guarantee of bug-free code within its **SMART CONTRACT**.

**ContractWolf**'s presence is to analyze, audit and assess the Client's Smart Contract to find any underlying risk and to eliminate any logic and flow errors within its code.

*Each company or project should be liable to its security flaws and functionalities.*

# SCOPE OF WORK | Veil

**Veil** team has agreed and provided us with the files that need to be tested (*Github, BSCscan, Etherscan, Local files etc*). The scope of audit is the main contract.

The goal of this engagement is to identify if there is a possibility of security flaws in the implementation of smart contract and its systems.

ContractWolf will be focusing on contract issues and functionalities along with the project claims from smart contract to their website, whitepaper, repository which has been provided by **Veil**.

# AUDITING APPROACH | Veil

Every line of code along with its functionalities will undergo manual review to check for security issues, quality of logic and contract scope of inheritance. The manual review will be done by our team that will document any issues that they discovered.

## METHODOLOGY

The auditing process follows a routine series of steps :

1. Code review that includes the following :
   - Review of the specifications, sources and instructions provided to ContractWolf to make sure we understand the size, scope and functionality of the smart contract.
   - Manual review of code. Our team will have a process of reading the code line-by-line with the intention of identifying potential vulnerabilities, underlying and hidden security flaws.

2. Testing and automated analysis that includes :
   - Testing the smart contract function with common test cases and scenarios to ensure that it returns the expected results.

3. Best practices and ethical review. The team will review the contract with the aim to improve efficiency, effectiveness, clarifications, maintainability, security and control within the smart contract.

4. Recommendations to help the project take steps to eliminate or minimize threats and secure the smart contract.

## TOKEN DETAILS | Veil



Veil is a privacy layer for Solana wallets, built to make private transactions a seamless part of everyday DeFi. This review covers the Veil browser extension, version 0.1.5: the wallet and its encrypted vault, the client that builds the zero knowledge proofs, and the shared app and SDK code the extension packages.

## SOURCE

| | |
|---|---|
| Source | *veil-extension -0.1.7* |

# FINDINGS | Veil



| 10 | 0 | 1 | 1 | 5 | 3 | 10 |
|---|---|---|---|---|---|---|
| Total Findings | Critical | Major | Medium | Minor | Informational | Resolved |

This report has been prepared to state the issues and vulnerabilities for Veil through this audit. The goal of this report findings is to identify specifically and fix any underlying issues and errors

| ID | Title | File & Line # | Severity | Status |
|---|---|---|---|---|
| CONSOLIDATE-FEE-UNBOUND | Note consolidation drops the reviewed fee token, so its ceiling is read in the wrong units | veil-sdk/src/client.ts, L: 479 | Major | Resolved |
| RELAYER-QUOTE-MIN | A private swap's minimum comes from the relayer that also builds the route | views/Forms.tsx, L: 591 | Medium | Resolved |
| UNLOCK-KEY-RESIDUE | An expired or orphaned unlock key is never removed from IndexedDB | lib/vault.ts, L: 93 | Minor | Resolved |
| VAULT-OVERWRITE | Restoring a recovery key overwrites an existing vault's records without warning | components/CreateWallet.tsx, L: 43 | Minor | Resolved |
| UNPRICED-FEE-CEILING | No relayer fee ceiling applies to a token without a USD price | lib/fees.ts, L: 13 | Minor | Resolved |
| RCW-013 | Every unlock re-downloads and re-scans the whole commitment set | veil-sdk/src/client.ts, L: 101 | Minor | Resolved |
| RCW-016 | The client Merkle tree keeps every leaf and rebuilds itself for each proof | veil-crypto/src/index.ts, L: 102 | Minor | Resolved |
| DEP-BIGINT-BUFFER | A bundled dependency carries an advisory with no fixed version | sbom.json | Informational | Resolved |
| LOCK-NO-ZEROIZE | Lock drops the references to the keys without clearing them | veil-sdk/src/client.ts, L: 102 | Informational | Resolved |

| | | | | |
|---|---|---|---|---|
| RANDFIELD-BIAS | Field elements are sampled by reducing 256 random bits | veil-crypto/src/index.ts, L: 26 | Informational | Resolved |

# RUST ATTACKS | Veil

Rust Smart Contract Weakness Classification and Test Cases

| ID | Name | Description | Status |
|---|---|---|---|
| RCW-001 | Reentrancy | A malicious contract calls back into the calling contract before the first invocation of the function is finished. | Passed |
| RCW-002 | Undefined behavior | The Rust reference contains a non-exhaustive list of behaviors considered undefined in Rust. | Passed |
| RCW-003 | Integer overflow & underflow | Overflow or underflow of mathematical operations inside the Rust smart contract. | Passed |
| RCW-004 | Out of bounds read/write | The contract or function reads data past the end, or before the beginning, of the intended buffer. | Passed |
| RCW-005 | Memory corruption | Wrong usage of the memory model throughout the contract or within its functions. | Passed |
| RCW-006 | Typographical error | Unintended error in contract and function names, code, and arithmetic inputs. | Passed |
| RCW-007 | Hash collisions with multiple arguments | If used incorrectly, triggers a hash collision while calling a function within a function. | Passed |
| RCW-008 | Account confusion | Failure to verify account ownership or types, leading to unauthorized access. | Passed |
| RCW-009 | Insufficient rent exemption | Accounts not properly funded for rent exemption, risking purge. | Passed |
| RCW-010 | Unverified PDA derivations | Not validating Program Derived Address (PDA) seeds or bump. | Passed |
| RCW-011 | Missing authority checks | Unauthorized token minting via unverified authorities, and improper freeze authorities. | Passed |
| RCW-012 | Arithmetic precision loss | Truncation errors in financial calculations and default round-offs (e.g. 0.999 â 0). | Passed |
| RCW-013 | Compute unit exhaustion | Unbounded loops causing transaction failures. | Passed |
| RCW-014 | Reward distribution flaws | Inaccurate reward pro-rata calculations. | Passed |
| RCW-015 | Slot-based race conditions | Front-running via transaction ordering. | Passed |
| RCW-016 | Uncontrolled allocations | Memory allocation without size limits. | Passed |
| RCW-017 | Erroneous computation | Incorrect or unintended math calculation. | Passed |
| RCW-018 | Function manipulation | Manipulation attack of time-difference values to increase rewards. | Passed |
| RCW-019 | Denial of service | DDoS attack using pending transactions. | Passed |
| RCW-020 | High verbosity logging | The product does not properly control the allocation and maintenance of a limited resource. | Passed |
| RCW-021 | Unrevoked Token Delegate | Verifies proper revocation of delegated token permissions after expiration time, to prevent unauthorized post-deadline transactions. | Passed |

| ID | Name | Description | Status |
|---|---|---|---|
| RCW-022 | Unverified vault authority in token swaps | Checks validation of swap vault ownership, to prevent malicious actors draining liquidity pools via unauthorized access. | Passed |
| RCW-023 | Solana BPF loader address spoofing | Tests detection of forged BPF program IDs, to block execution of malicious smart contracts masquerading as trusted programs. | Passed |
| RCW-024 | Missing signer checks | Instructions that change state or move funds without confirming the required account actually signed the transaction. | Passed |
| RCW-025 | Arbitrary cross-program invocation | Calling a program ID supplied by the caller instead of a verified one, handing control to attacker-written code. | Passed |
| RCW-026 | Duplicate mutable accounts | The same account passed twice under different names, letting one instruction overwrite its own state or bypass balance checks. | Passed |
| RCW-027 | Account data reinitialization | Initialization that can be run again on a live account, resetting authorities or balances to attacker-chosen values. | Passed |
| RCW-028 | Account closing and revival | Closed accounts whose lamports are drained but data is not zeroed, so they can be revived within the same transaction. | Passed |
| RCW-029 | Type cosplay | Account data deserialized without a discriminator check, so one account type is accepted in place of another. | Passed |
| RCW-030 | Sysvar spoofing | Sysvar accounts such as Clock or Instructions read from caller-supplied accounts without verifying their addresses. | Passed |
| RCW-031 | Bump seed canonicalization | PDAs accepted with non-canonical bump seeds, producing multiple valid addresses for one logical account. | Passed |
| RCW-032 | Seed collisions | PDA seed schemes where different inputs concatenate to the same address, merging separate users' state. | Passed |
| RCW-033 | Unsafe Rust blocks | Use of unsafe code, raw pointers and transmute that bypass the compiler's memory and type guarantees. | Passed |
| RCW-034 | Panics and unchecked unwrap | unwrap, expect, indexing and division that abort the program on attacker-controlled input and block critical paths. | Passed |
| RCW-035 | Unsafe numeric casts | Truncating or sign-changing casts between integer widths that silently alter amounts. | Passed |
| RCW-036 | Token-2022 extension handling | Transfer fees, hooks, and permanent delegate extensions that break assumptions about transferred amounts and custody. | Passed |
| RCW-037 | Stale account data after CPI | Reading a balance or state value cached before a cross-program invocation that has since changed it. | Passed |
| RCW-038 | Oracle price manipulation | Reliance on a single spot price or stale, low-confidence oracle feed that can be moved or outdated. | Passed |
| RCW-039 | Upgrade authority risk | Programs left upgradeable under a single key, or with the authority misconfigured, allowing code replacement. | Passed |
| RCW-040 | Rounding direction errors | Rounding that favors the user instead of the protocol on deposits, withdrawals and fees, enabling dust extraction. | Passed |
| RCW-041 | Instruction introspection abuse | Checks on sibling instructions in the transaction that can be satisfied by crafted ordering or padding. | Passed |
| RCW-042 | Vulnerable dependencies | Crates in Cargo.lock with known advisories, unpinned versions, or unaudited forks pulled into the build. | Passed |

## FIXES & RECOMMENDATION

### CONSOLIDATE-FEE-UNBOUND | Note consolidation drops the reviewed fee token, so its ceiling is read in the wrong units

`withConsolidation()` hands `consolidate()` only `maxFee`, dropping the `feeMint` the user approved. `planTransfer()` falls back to paying the relayer in the spend token whenever private SOL cannot cover the quoted SOL fee, and `hold()` then compares that token fee against a ceiling expressed in lamports. A relayer that quotes a high SOL fee forces the fallback and may charge up to the lamport figure in base units of the other token.

```
// client.ts, consolidate(): the fee token the user reviewed is dropped
return this.transfer(this.k().address!, ref, total, onProgress, notes,
  limits && { maxFee: limits.maxFee });

// client.ts, hold(): without feeMint, nothing checks which token the fee is in
private hold(limits: SpendLimits | undefined, fee: bigint, feeAsset: AssetInfo,
             hidden?: boolean) {
  if (!limits) return;
  if (limits.feeMint && limits.feeMint !== feeAsset.mint.toBase58()) throw new Error(...);
  if (limits.maxFee != null && fee > limits.maxFee) throw new Error(...);
}
```

**Recommendation**

Carry the whole `SpendLimits` into `consolidate()` and keep a ceiling per fee asset rather than one number, so a merge whose `feeAsset` is not the approved one stops and is reviewed again.

```
// client.ts: a ceiling per asset, and the merge is held to the same terms
export interface SpendLimits {
  maxFeeByMint?: Record<string, bigint>;
  feeMint?: string;
  hidden?: boolean;
}

private hold(limits: SpendLimits | undefined, fee: bigint, feeAsset: AssetInfo,
             hidden?: boolean) {
  if (!limits) return;
  const mint = feeAsset.mint.toBase58();
  const cap = limits.maxFeeByMint?.[mint];
  if (cap == null) throw new Error('The relayer fee would be paid in a token you '
    + 'did not review. Nothing was sent; review it again.');
  if (fee > cap) throw new Error('The relayer fee went up after you reviewed it.');
  if (limits.hidden && hidden === false) throw new Error('This send would now show '
    + 'its token on-chain. Nothing was sent; review it again.');
}

async consolidate(ref: AssetRef, onProgress?: Progress, limits?: SpendLimits) {
  // the merge is held to every term the spend itself was held to
  return this.transfer(this.k().address!, ref, total, onProgress, notes, limits);
}
```

## RELAYER-QUOTE-MIN | A private swap's minimum comes from the relayer that also builds the route

`Trade` sets `minOut` from `v.quote()`, which is the relayer's own number, and the same relayer picks the route the program executes. The only independent check is gap, computed from catalogue prices that Veil's own indexer serves, and it blocks only a quote more than 5% under market. When either token has no `usdPrice` the check is skipped and any quote is accepted.

```
// Forms.tsx: the quote, the minimum and the route all come from the relayer
const minOut = q != null ? (q * BigInt(10_000 - slip)) / 10_000n : null;
const market = tok && out && a && tok.usdPrice && out.usdPrice
  ? ((Number(a - spfee) / 10 ** tok.decimals) * tok.usdPrice) / out.usdPrice
  : null;
const gap = market && q != null && market > 0
  ? 1 - Number(q) / 10 ** out!.decimals / market
  : null;
// gap === null (no price on either side) falls through and the trade proceeds
const block = ... : gap != null && gap > 0.05 ? 'Quote is under the market price' : '';
```

### Recommendation

Price the pair against a source outside Veil's own services before fixing `minOut`, refuse a pair with no independent price instead of warning, and narrow the 5% gate toward the slippage the user chose.

```
// Forms.tsx: no trade without an independent price, and a tighter gate
const independent = await priceFromOutsideVeil(tok.key, out.key);   // not the relayer
const market = independent && a
  ? (Number(a - spfee) / 10 ** tok.decimals) * independent
  : null;
const gap = market && q != null && market > 0
  ? 1 - Number(q) / 10 ** out!.decimals / market
  : null;
// the minimum follows the independent price, never the relayer's quote
const minOut = market != null
  ? (BigInt(Math.floor(market * 10 ** out!.decimals)) * BigInt(10_000 - slip)) / 10_000n
  : null;
const block = market == null
  ? 'No independent price for this pair: this trade cannot be checked'
  : gap != null && gap > slip / 10_000 + 0.01
  ? 'The quote is below the market price: try a smaller amount'
  : '';
```

## UNLOCK-KEY-RESIDUE | An expired or orphaned unlock key is never removed from IndexedDB

`rememberUnlock()` keeps the vault's AES key in `veil/unlocked` with an `until` field, but nothing deletes it when that moment passes: `unlockRemembered()` only drops it on the next read, which may never come. `resetWallets()` rewrites the wallet list after a restore, so `forgetAll()`, which walks that list, can no longer reach the keys of the wallets it replaced.

```
// vault.ts: the expiry is only ever enforced when the row is read
export async function rememberUnlock(v: OpenVault, days: number) {
  await tx('readwrite', s => {
    s.put({ key: v.key, until: Date.now() + days * 86_400_000 }, v.rec.wallet);
  }, 'unlocked');
}

// wallets.ts: Lock only clears the wallets still on the list
export async function forgetAll() {
  lockGen++;
  for (const w of await read()) await Vault.forgetUnlock(w.entry);
  const a = await getEntry(); if (a) await Vault.forgetUnlock(a);
}

// wallets.ts: a restore replaces the list, orphaning every other wallet's key
export async function resetWallets(entry: string, deposit: string) {
  await write([{ entry, deposit, name: 'Wallet 1', created: Date.now(), from: 'main' }]);
}
```

**Recommendation**

Sweep every `veil/unlocked` row past its `until` when the extension starts and on each lock, and delete the vault and the remembered key of each wallet `resetWallets()` drops before it writes the new list.

```
// vault.ts: drop every expired key, whether or not its wallet is read again
export async function sweepExpired() {
  const db = await open(); if (!db) return;
  await new Promise<void>(res => {
    const t = db.transaction('unlocked', 'readwrite');
    const s = t.objectStore('unlocked'), q = s.openCursor();
    q.onsuccess = () => {
      const c = q.result; if (!c) return;
      if ((c.value as Remembered).until < Date.now()) c.delete();
      c.continue();
    };
    t.oncomplete = () => res(); t.onerror = () => res();
  });
}

// wallets.ts: nothing is left behind by a restore
export async function resetWallets(entry: string, deposit: string) {
  for (const w of await read()) {
    if (w.entry === entry) continue;
    await Vault.forgetUnlock(w.entry);
    await Vault.deleteVault(w.entry);
  }
  await Vault.sweepExpired();
  await write([{ entry, deposit, name: 'Wallet 1', created: Date.now(), from: 'main' }]);
}
```

## VAULT-OVERWRITE | Restoring a recovery key overwrites an existing vault's records without warning

`make()` and `seal()` both call `createVault()`, which writes a fresh payload over whatever is already stored for that wallet address. The import screen asks before replacing the imported wallet itself, but `discover()` rewrites every derived wallet it finds on chain in silence. Activity, the one-time address list and open `BridgeOrder` records, including the signed quote kept to settle a dispute, are lost.

```
// CreateWallet.tsx: a restore seals a brand new, empty payload over the old one
const make = async (root: Uint8Array, fresh: boolean) => {
  const entry = entryKeypair(root).publicKey.toBase58();
  const v = await Vault.createVault(entry, pass, root);
  if (fresh) await Vault.saveVault(v, { cost: { v: 1, t: {} } });
  await Vault.rememberUnlock(v, 1 / 96);
  await resetWallets(entry, depositAddress(root).toBase58());
  return entry;
};

// wallets.ts, discover(): every wallet found on chain is sealed the same way,
// with no prompt and nothing carried over from what was stored for it
const sealed = await seal(root, pass, idleMinutes, false);
```

**Recommendation**

Read any record already stored for the address and carry `activity`, `stealth` and `bridges` into the new payload, and warn for each wallet that would be rewritten, naming any bridge order still in flight.

```
// vault.ts: a restore keeps what the old record held, under the new password
export async function reseal(wallet: string, pass: string, root: Uint8Array,
                             old: OpenVault | null): Promise<OpenVault> {
  const v = await createVault(wallet, pass, root);
  if (!old) return v;
  await saveVault(v, {
    activity: old.activity,
    stealth: old.stealth,
    bridges: old.bridges,
    ...(old.cost ? { cost: old.cost } : {})
  });
  return v;
}

// CreateWallet.tsx: name what would be rewritten before anything is written
const open = existing ? await Vault.unlockVault(existing, oldPass).catch(() => null) :
null;
const inFlight = (open?.bridges ?? []).filter(b => b.status !== 'SUCCESS');
if (inFlight.length && !replace) throw new Error(`${inFlight.length} bridge order(s) `
  + 'are still running on this wallet. Press Import again to go ahead.');
const v = await Vault.reseal(entry, pass, root, open);
```

## UNPRICED-FEE-CEILING | No relayer fee ceiling applies to a token without a USD price

`feeTooHigh()` checks `FEE_CAP_SOL` only for wrapped SOL and `FEE_CAP_USD` only when `usdPrice` is set, so for any other token it returns `false` whatever the fee is. Send, withdraw, trade, earn and the manual shield all gate on it, so for an unpriced token a relayer can quote a fee near the whole amount and only the printed figure stands in the way.

```
// fees.ts: an unpriced, non-SOL token never trips the ceiling
const FEE_CAP_SOL = 20_000_000n, FEE_CAP_USD = 5;

export const feeTooHigh = (t: Priced | undefined, fee: bigint | null | undefined) =>
  !!t && fee != null && (t.mint.equals(NATIVE_MINT)
    ? fee > FEE_CAP_SOL
    : t.usdPrice != null
      && (Number(fee) / 10 ** t.decimals) * t.usdPrice > FEE_CAP_USD);
```

### Recommendation

Fall back to a share of the amount when a token has no price, the way `AutoShield` already does with `MAX_FEE_SHARE`, and require an explicit confirmation for anything above that share.

```
// fees.ts: every token has a ceiling, priced or not
const FEE_CAP_SOL = 20_000_000n, FEE_CAP_USD = 5;
/** Without a price, a fee may not take more than this share of the amount. */
const FEE_CAP_SHARE = 20n;   // 1/20 = 5%

export const feeTooHigh = (t: Priced | undefined, fee: bigint | null | undefined,
                           amount?: bigint | null) => {
  if (!t || fee == null) return false;
  if (t.mint.equals(NATIVE_MINT)) return fee > FEE_CAP_SOL;
  if (t.usdPrice != null) return (Number(fee) / 10 ** t.decimals) * t.usdPrice >
FEE_CAP_USD;
  // no price: hold the fee to a share of what is moving, and ask before going over
  return amount == null || amount <= 0n || fee * FEE_CAP_SHARE > amount;
};
```

## RCW-013 | Every unlock re-downloads and re-scans the whole commitment set

`unlock()` clears the tree and resets synced to 0, so `sync()` walks every commitment the indexer holds from index 0 and runs an X25519 shared secret and an AES-GCM open on each ciphertext. Nothing is kept between sessions. The work grows with the global anonymity set rather than with the wallet, inside a popup Chrome may stop at any time.

```
// client.ts: opening a vault throws away everything scanned before
unlock(keys: V.VeilKeys) {
  this.keys = keys;
  this.notes.clear();
  this.tree = new V.MerkleTree();
  this.synced = 0;
}

// client.ts, sync(): one key agreement and one AES-GCM open per commitment,
// for every commitment ever made, on every unlock
for (const c of commitments) {
  this.tree.insert(V.fromBytes(V.unhex(c.commitment)));
  const enc = c.encryptedOutput ? V.unhex(c.encryptedOutput) : null;
  if (enc && enc.length) {
    const d = await V.openNote(keys.vsk, enc);
    ...
  }
}
```

**Recommendation**

Keep the scanned leaf index, the tree frontier and the notes found in the sealed vault payload and resume from there, and bound a single sync so the rest continues in the background.

```
// client.ts: resume a scan instead of starting it over
export interface ScanState {
  synced: number;
  frontier: string[];          // the filled nodes, one per level
  notes: [number, OwnedNote][];
}

unlock(keys: V.VeilKeys, saved?: ScanState) {
  this.keys = keys;
  this.notes = new Map(saved?.notes ?? []);
  this.tree = saved ? V.MerkleTree.fromFrontier(saved.frontier) : new V.MerkleTree();
  this.synced = saved?.synced ?? 0;
}

/** Scans at most `budget` leaves, then returns so the page stays responsive. */
async syncSome(budget = 20_000): Promise<{ done: boolean; state: ScanState }> {
  const start = this.synced;
  while (this.synced - start < budget) {
    const { commitments } = await this.indexer.commitments(this.synced, 5_000);
    if (!commitments.length) return { done: true, state: this.scanState() };
    await this.applyPage(commitments);
  }
  return { done: false, state: this.scanState() };
}
```

## RCW-016 | The client Merkle tree keeps every leaf and rebuilds itself for each proof

`MerkleTree` holds every commitment in `leaves`, and `path()` copies that whole array and recomputes each level to produce a single Merkle path, once per input note. A 26 level tree admits tens of millions of leaves, so memory and the time to set a proof up grow with no bound at all, inside a popup with no say over how much it may use.

```
// veil-crypto/src/index.ts: every leaf is kept, and a path rebuilds the whole tree
export class MerkleTree {
  leaves: bigint[] = [];
  private filled = ZEROS.slice(0, DEPTH);
  root = ZEROS[DEPTH];

  path(index: number): bigint[] {
    let level = this.leaves.slice(), idx = index; const out: bigint[] = [];
    for (let l = 0; l < DEPTH; l++) {
      const sib = idx ^ 1; out.push(sib < level.length ? level[sib] : ZEROS[l]);
      const next: bigint[] = [];
      for (let i = 0; i < level.length; i += 2)
        next.push(poseidon2([level[i], i + 1 < level.length ? level[i + 1] : ZEROS[l]]));
      level = next; idx >>= 1;
    }
    return out;
  }
}
```

**Recommendation**

Hold only the frontier and the sibling nodes a proof needs, fetch each path from the indexer and check it against the root the program itself recorded, and cap how many leaves stay in memory.

```
// veil-crypto/src/index.ts: the frontier alone, and paths checked against the root
export class MerkleTree {
  count = 0;
  private filled = ZEROS.slice(0, DEPTH);
  root = ZEROS[DEPTH];

  /** Inserts a leaf keeping only the frontier: memory stays at DEPTH nodes. */
  insert(leaf: bigint): number {
    let idx = this.count, cur = leaf;
    for (let l = 0; l < DEPTH; l++) {
      if ((idx & 1) === 0) { this.filled[l] = cur; cur = poseidon2([cur, ZEROS[l]]); }
      else cur = poseidon2([this.filled[l], cur]);
      idx >>= 1;
    }
    this.root = cur;
    return this.count++;
  }
}

/** A path served by the indexer is used only once it rebuilds a root we trust. */
export function checkedPath(leaf: bigint, index: number, path: bigint[],
                            trustedRoot: bigint): bigint[] {
  if (rootFromPath(leaf, index, path) !== trustedRoot)
    throw new Error('the path served does not rebuild the on-chain root');
  return path;
}
```

### DEP-BIGINT-BUFFER | A bundled dependency carries an advisory with no fixed version

`sbom.json` lists `bigint-buffer@1.1.5`, reached through `@solana/buffer-layout-utils`. Its published advisory covers an out of bounds write in the native `toBigIntLE()` addon, and no fixed release exists. A browser bundle loads no native addon, so the pure JavaScript fallback runs and the flaw is not reachable here, but the package stays in the dependency graph the build shares with the services.

#### Recommendation

Add a resolution override that removes `bigint-buffer`, or replace `@solana/buffer-layout-utils` with `DataView` based encoding, so the advisory does not follow this dependency graph into a Node build.

## LOCK-NO-ZEROIZE | Lock drops the references to the keys without clearing them

`lock()` sets `keys` to null and `VeilProvider.lock()` drops the vault from state, but the 32 byte root, the sk and nk field elements and the decrypted payload stay in the heap until the collector runs, and a `BigInt` cannot be overwritten in place. The stated goal is that a lock leaves no usable key behind in memory.

```
// client.ts: the reference goes, the bytes stay
lock() { this.keys = null; this.notes.clear(); }

// VeilProvider.tsx: the root is dropped from React state, never cleared
lock() {
  if (publicKey) void Vault.forgetUnlock(publicKey.toBase58());
  setVault(null); setActivity([]); setBridges([]); setCost(null);
  setPriv({}); setLastSync(0); setRootStatus('unknown');
  clientRef.current?.lock();
}
```

### Recommendation

Overwrite the root and every secret byte array with zeros before dropping them, hold field elements as `Uint8Array` where the maths allows it, and state the residual limit in the security notes.

```
// vault.ts: what can be erased, is
export function wipe(v: OpenVault | null) {
  if (!v) return;
  v.root.fill(0);
  v.keys.vsk.fill(0);
  v.keys.vpk.fill(0);
  // sk and nk are BigInt and cannot be overwritten: drop the only reference
  (v.keys as { sk?: bigint; nk?: bigint }).sk = undefined;
  (v.keys as { sk?: bigint; nk?: bigint }).nk = undefined;
}

// VeilProvider.tsx
lock() {
  if (publicKey) void Vault.forgetUnlock(publicKey.toBase58());
  clientRef.current?.lock();
  Vault.wipe(vault);
  setVault(null); setActivity([]); setBridges([]); setCost(null);
  setPriv({}); setLastSync(0); setRootStatus('unknown');
}
```

## RANDFIELD-BIAS | Field elements are sampled by reducing 256 random bits

`randField()` and the note blinding both reduce a 256 bit value modulo P, which is close to 2 to the 253.5, so the lowest residues come up slightly more often than the rest. The bias costs a fraction of a bit of entropy and is not exploitable at this size, but it is a departure from uniform sampling in a value whose job is to hide a note.

```
// veil-crypto/src/index.ts
export const randField = () => fromBytes(rand(32)) % P;

/** The blinding of a sealed note is reduced the same way. */
const derivedBlinding = (shared: Uint8Array, epk: Uint8Array) =>
  fromBytes(kdf(shared, epk, 'blinding')) % P;
```

### Recommendation

Draw again whenever the 256 bit value is at or above the largest multiple of P below 2 to the 256, or reduce from a wider buffer, so every field element is equally likely.

```
// veil-crypto/src/index.ts: rejection sampling, so the draw is uniform
const LIMIT = (1n << 256n) - ((1n << 256n) % P);   // largest multiple of P that fits

export const randField = () => {
  for (;;) {
    const x = fromBytes(rand(32));
    if (x < LIMIT) return x % P;
  }
};

/** A derived blinding has no second draw, so it is widened instead. */
const derivedBlinding = (shared: Uint8Array, epk: Uint8Array) =>
  fromBytes(cat(kdf(shared, epk, 'blinding'), kdf(shared, epk, 'blinding2'))) % P;
```

# AUDIT COMMENTS | Veil

Smart Contract audit comment for a non-technical perspective

- Veil does not record or collect any IP addresses from its users
- Nobody at Veil can move, freeze or block a private balance. Spending needs the vault's own key and a proof built in the browser, so a relayer that refuses to relay can delay a user but never redirect value.
- The relayer sets its own fee for every private action. The app refuses a fee above a ceiling the user has reviewed, but that ceiling is not enforced for a token with no known price.
- Add funds shields whatever arrives at the deposit address by itself, but only while a Veil page is open and unlocked, and it is skipped whenever the fee would take more than 5% of the deposit.
- There is no account recovery. Losing both the password and the recovery key loses the private balance permanently, and anyone who obtains the recovery key can spend it.
- The on-chain program this wallet talks to is governed by Veil's own admin and upgrade authority. That program is outside this review(veil-main/core) and was audited separately on October 6, 2026.



# CONTRACTWOLF

Blockchain Security - Smart Contract Audits